

EchoMind TRD

The Organizational Memory Operating System

Version: 1.0
Status: Draft / Hackathon Ready
Project: Japan-India Innovation Hackathon
Prepared By: Team EchoMind

1. System Architecture Overview

EchoMind follows a modern, decoupled microservices architecture designed to handle both unstructured data ingestion and high-speed semantic retrieval.

The architecture is divided into three primary layers:

- 1. **Client Layer:** A responsive web application.
- 2. **Application Layer:** Backend microservices routing traffic, handling business logic, and orchestrating AI workflows.
- 3. **Data & AI Layer:** A tri-database system (Relational, Graph, and Vector) powering the AI Reasoning Extractor and Recommendation Engine.

2. Proposed Technology Stack

Component	Technology	Rationale
Frontend UI	Next.js (React), Tailwind CSS, D3.js	Next.js for SSR and fast routing; D3.js for rendering the interactive Knowledge Graph timeline.
Backend API	FastAPI (Python)	High performance, native async support, and seamless integration with Python-based AI/ML libraries.
Primary Database	PostgreSQL	Stores user profiles, RBAC configurations, metadata, and audit logs.
Graph Database	Neo4j	Handles the complex node-edge relationships of the Organizational Knowledge Graph.

Component	Technology	Rationale
Vector Database	Pinecone <i>or</i> Milvus	Stores document and reasoning embeddings for fast semantic similarity search.
AI / NLP Framework	LangChain / LlamaIndex	Orchestrates LLM prompt chains, data chunking, and Retrieval-Augmented Generation (RAG).
LLM Provider	OpenAI API (GPT-4o) <i>or</i> Claude 3.5	Best-in-class extraction of structured reasoning from unstructured text.

3. Data Architecture & Modeling

EchoMind relies on a hybrid data storage strategy to link raw text, semantic meaning, and structural relationships.

3.1 The Knowledge Graph (Neo4j)

Data is modeled as nodes and edges to reflect the **EchoMind Decision Framework**.

- **Nodes:** `User` , `Machine/Asset` , `Symptom` , `Analysis` , `Decision` , `Outcome` .
- **Edges (Relationships):** `REPORTED_BY` , `HAS_SYMPTOM` , `LED_TO_DECISION` , `RESULTED_IN` .

3.2 Vector Similarity Engine

To power the **Similar Case Finder**, textual descriptions of problems and decisions are converted into dense vector embeddings. The system computes similarity using the cosine similarity mathematical model:

$$\text{cosine_similarity}(u, v) = (u \cdot v) / (||u|| \ ||v||)$$

Where *u* is the embedding vector of the new user query, and *v* represents the historical cases in the vector database.

4. Core System Modules & APIs

4.1 Decision Capture API (`/api/v1/capture`)

- **Function:** Accepts manual form inputs (text, voice transcripts).
- **Process:** Validates payload → stores raw data in PostgreSQL → triggers async job to create Graph Nodes.

4.2 AI Reasoning Extractor Pipeline

- **Function:** Ingests unstructured files (.pdf, .txt) and outputs structured graph data.
- **Workflow:**
 1. **Parsing:** Extract text using OCR / PDF parsers.

2. **Chunking:** Split text into overlapping semantic chunks.
3. **LLM Extraction:** Prompt the LLM to identify the *Situation*, *Analysis*, *Options*, *Decision*, and *Outcome*.
4. **Graph Insertion:** Write the extracted relationships to Neo4j.
5. **Embedding:** Generate embeddings for the "Situation" and "Decision" and store them in the Vector DB.

4.3 Recommendation Engine API (`/api/v1/recommend`)

- **Function:** Handles natural language queries.
- **Workflow:** User Query → Embedding Generation → Vector DB Semantic Search → Graph DB Traversal → LLM Synthesis → Client Response.

5. Security & Privacy Architecture

- **Authentication & Authorization:** OAuth 2.0 / OpenID Connect; JWT (JSON Web Tokens) for stateless API authentication.
- **Data Encryption:** TLS 1.3 enforced for in-transit data; AES-256 encryption for data at rest.
- **Data Anonymization Pipeline:** Implement NLP scrubbing to detect and mask PII before saving to the Vector DB or Knowledge Graph.

6. Scalability & Performance Parameters

- **Graph Traversal:** Queries must return historical paths within < 500 ms.
- **Vector Search:** ANN search over 100,000+ vectors must execute in < 200 ms.
- **Caching:** Implement Redis to cache frequent queries.
- **Rate Limiting:** Apply token-bucket rate limiting at the API Gateway to prevent abuse.

7. Deployment Strategy (Hackathon Environment)

- **Containerization:** All services packaged using Docker (`docker-compose`).
- **Cloud Hosting:** Deploy the backend APIs and Frontend on Vercel or AWS.
- **Managed Databases:** Use Neo4j AuraDB, Supabase, and Pinecone Serverless to minimize DevOps overhead.